INDIAN INSTITUTE OF TECHNOLOGY BHILAI
Department of Computer Science & Engineering

Systems Architecture and Empirical Evaluation for Statistical NLP, Parsing, and Live-Typing Analysis

DSL504 — Natural Language Processing
Group Assignment 1 | Group 14

Kinshuk Gupta	12341190
Harshit Kandpal	B24CS505
Sujal Som	B24DS032
Utkarsh Chatterjee	B24CS050
Tanmay Jaiswal	B24DS033

{kinshukg,harshitka,sujals,utkarshchatt,tanmay}@iitbhilai.ac.in

Repository & Live Deployment

Source Code: https://github.com/HarK-github/group_assignment_1_group14
Live Application: https://groupassignment1group14-yiipvtgjukqr9x9fnirabf.streamlit.app/

September 10, 2026

Abstract

This report presents the design, implementation, and rigorous empirical evaluation of a four-part statistical Natural Language Processing (NLP) system developed for DSL504 Course. The system spans (i) dynamic-programming word segmentation and Hidden Markov Model (HMM) part-of-speech tagging with morphology-aware extensions for English and Spanish, (ii) an Arc-Standard transition-based dependency parser trained via oracle simulation on Universal Dependencies English-EWT, (iii) a dual-algorithm spelling correction engine contrasting exhaustive edit-distance-1 generation against a symmetric-delete lookup structure, and (iv) a fully integrated Streamlit-based live-typing editor that fuses all preceding components with a PCFG-based constituency parser and a tiered n-gram grammaticality decision rule. Across 100 held-out sentences per language, the trigram dynamic-programming segmenter improves boundary-level F1 from 0.729 (greedy baseline) to 0.931 for English and from 0.666 to 0.896 for Spanish, while the HMM tagger attains 92.97% and 89.12% accuracy respectively. The dependency parser, trained on 407,165 oracle-derived transition instances across 82 action classes, achieves 66.70% Unlabeled Attachment Score (UAS) and 56.71% Labeled Attachment Score (LAS) at a throughput of 140.9 sentences/second. The symmetric-delete spelling corrector delivers an 11.5× latency reduction over exhaustive edit-distance generation while maintaining 52.60% non-word and 67.91% real-word correction accuracy. Finally, the integrated editor sustains end-to-end per-token processing latencies well within the 16 ms interactive rendering budget, confirming the architecture's suitability for real-time deployment. All source code, trained model artifacts, and a live demonstration are made publicly available.

Keywords: Statistical NLP, Hidden Markov Models, Transition-Based Dependency Parsing, Symmetric-Delete Spelling Correction, Probabilistic Context-Free Grammars, Real-Time Text Editing.

Contents

List of Tables

List of Figures

1 Introduction

1.1 System Design Philosophy

The overall design is intentionally modular and classical: every component is grounded in dynamic programming, sparse feature classification, corpus-frequency estimation, or fast probabilistic lookup, rather than end-to-end neural modeling. **Question 1** trains trigram language models and Viterbi-style dynamic-programming segmenters for English and Spanish, and layers an HMM part-of-speech (POS) tagger with a morphology-aware extension for richer agreement modeling. **Question 2** implements a transition-based Arc-Standard dependency parser trained from oracle-generated state-action pairs over CoNLL-U formatted data. **Question 3** builds a Brown-corpus spelling correction engine that contrasts exhaustive edit-distance-1 candidate generation against a symmetric-delete lookup structure. **Question 4** integrates the three preceding components into a Streamlit-based background editor with live segmentation, spelling, and grammar alerts, augmented with sentence-level analysis using a Probabilistic Context-Free Grammar (PCFG) alongside trigram and bigram scoring.

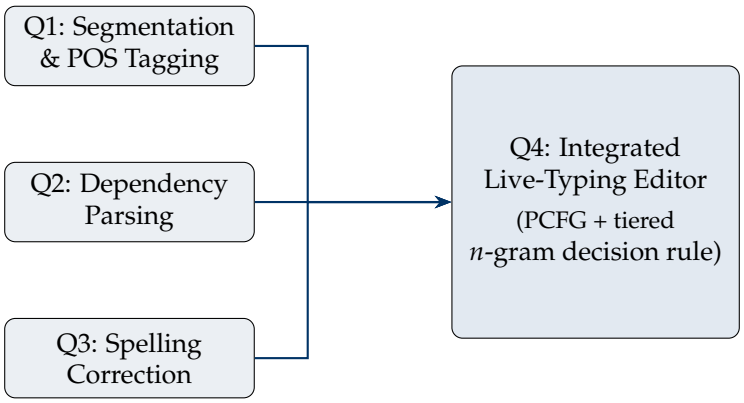

Figure 1.1: High-level system architecture. Q1 (segmentation/tagging) and Q3 (spelling correction) supply trained artifacts directly to the Q4 runtime pipeline; Q2 constitutes an independent parsing subsystem evaluated on Universal Dependencies.

1.2 Report Organization

Section 2 details word segmentation and POS tagging; Section 3 details the transition-based dependency parser; Section 4 details the spelling correction engine; Section 5 details the integrated live-typing editor; Section 6 provides a full reproducibility guide; Section 7 concludes; and the appendices enumerate the serialized model registry and reference code usage patterns.

2 Question 1: Word Segmentation and Part-of-Speech Tagging

2.1 Core Theory and Implementation Surface

The objective is to accurately perform word segmentation and POS tagging on unspaced text. Segmentation is driven by a unigram/bigram/trigram language model coupled to a dynamic-programming (DP) search mirroring Viterbi decoding. Smoothing is applied via an add-k strategy ($k = 0.05$), with backoff from trigram to bigram to unigram probabilities under sparse context counts. The `TrigramDPSegmenter` optimizes over unspaced strings using beam pruning (`beam_width=20`, `max_word_len=20`), imposing an out-of-vocabulary (OOV) penalty of -18.0 to regularize spurious single-character unknown segments.

The POS tagging module is a first-order HMM capturing both word–tag emission probabilities and tag–tag transition probabilities. Add-k smoothing ($k = 10^{-4}$) protects against zero-probability transitions while preserving dominant corpus distributions.

To contextualize the DP model's advantages, two baselines were constructed: a *greedy longest-match* segmenter and a *most-frequent-tag* (MFT) baseline. The greedy segmenter scans forward and matches the longest vocabulary string at each offset, providing only local foresight and isolating the exact value contributed by global DP sequence optimization.

2.2 Data and Language Coverage

The pipeline processes both English and Spanish. English uses the NLTK Brown corpus with an 80/20 train/test split, yielding 45,727 training sentences and 11,432 test sentences. Spanish uses the UD Spanish-GSD corpus, split into 14,186 train, 1,400 dev, and 427 test sentences (cached locally under `Question_1/ud_cache` and fetched from GitHub when absent). The extracted vocabularies comprise 44,145 English words and 41,638 Spanish words. English POS tagging resolves to 11 universal tags, while the standard Spanish branch models 16 base tags.

2.3 Morphology-Aware Modeling

Plain POS tags are insufficient for capturing grammatical constraints such as gender and number agreement. The Spanish tag inventory was therefore extended to compound representations combining the UPOS tag with the `Gender` and `Number` morphological features from the UD annotation, resulting in **85** compound tags. This design forces the HMM to explicitly model agreement constraints rather than collapsing them into a coarse universal label.

2.4 Empirical Evaluation

Table 2.1 reports boundary-level segmentation metrics over 100 held-out test sentences per language, isolating the DP algorithm's exact boundary-matching capability against the greedy baseline.

Table 2.1: Segmentation Evaluation Metrics (100 test sentences per language).

Setting	Precision	Recall	F1 Score	Exact Match
English Greedy	0.7420	0.7258	0.7292	15.0%
English DP	**0.9388**	**0.9272**	**0.9311**	**58.0%**
Spanish Greedy	0.6980	0.6480	0.6658	4.0%
Spanish DP	**0.9085**	**0.8902**	**0.8961**	**17.0%**

Table 2.2 compares the HMM tagger against the MFT baseline. The HMM consistently outperforms MFT, by 1.30 percentage points for English and 1.68 points for Spanish.

Table 2.2: POS Tagging Accuracy: MFT Baseline vs. HMM.

Language	MFT Accuracy	HMM Accuracy
English	91.67%	**92.97%**
Spanish	87.44%	**89.12%**

The compounded, end-to-end segmentation-plus-tagging accuracies are reported in Table 2.3, and confirm the substantial cumulative benefit of dynamic-programming segmentation prior to tagging.

Table 2.3: End-to-End (Segmentation + Tagging) Accuracy.

Language	Greedy + Baseline	DP + HMM
English	63.85%	**84.11%**
Spanish	53.42%	**75.13%**

An aligned error-source attribution analysis reveals that **72.25%** of English DP tagging errors and **80.34%** of Spanish DP tagging errors originate from upstream segmentation failures rather than tagging failures — a critical finding indicating that residual system error is dominated by boundary detection, not label assignment, even after DP optimization.

2.5 Confusion Matrix Analysis

Full confusion matrices were generated for both languages. The English matrix shows the heaviest diagonal mass concentrated on `NOUN` and `VERB`, with frequent confusions among `NOUN`, `VERB`, `DET`, `ADP`, `PRON`, `ADV`, `ADJ`, and `PRT`. The Spanish matrix similarly shows strong diagonal performance for `NOUN`, `ADP`, `DET`, `VERB`, `PROPN`, `ADJ`, `ADV`, and `PRON`, with **proper nouns (`PROPN`)** proving the most error-prone high-frequency category — consistent with the general difficulty of distinguishing proper nouns from common nouns under limited lexical context.

2.6 Morphological Agreement Analysis

The utility of morphology-aware tagging is decisively demonstrated by conditional agreement probabilities extracted from the trained Spanish morphology HMM:

$$P(\text{ADJ-Fem-Sing} \mid \text{NOUN-Fem-Sing}) = 0.0932, \quad P(\text{ADJ-Masc-Sing} \mid \text{NOUN-Fem-Sing}) = 0.0029 \quad (\text{ratio} \approx$$
$$P(\text{ADJ-Masc-Sing} \mid \text{NOUN-Masc-Sing}) = 0.0911, \quad P(\text{ADJ-Fem-Sing} \mid \text{NOUN-Masc-Sing}) = 0.0008 \quad (\text{ratio} \approx$$

These ratios provide direct empirical evidence that the morphology-aware tagger internalizes gender-agreement constraints that a standard universal tagset would otherwise flatten and discard.

2.7 Hyperparameter Configuration and Implementation Notes

Table 2.4 summarizes the hardcoded configuration governing the Question 1 pipeline.

Table 2.4: Question 1 Hardcoded Configuration Parameters.

Parameter	Value	Governs
Train/test split	80/20	Brown corpus partitioning
Trigram LM smoothing	$k = 0.05$	`TrigramLanguageModel` add-k estimate
Beam width	20	`TrigramDPSegmenter` beam pruning
Max word length	20	`TrigramDPSegmenter` candidate span cap
OOV penalty	-18.0	Penalizes single-character unknown segments
HMM smoothing	$k = 10^{-4}$	Transition/emission add-k estimate
Evaluation sample	100 sentences	Per-language segmentation/tagging test set

Implementation Notes

The Spanish data loader caches UD-GSD files under `Question_1/ud_cache` and transparently fetches them from GitHub when absent. For demonstration purposes, the notebook additionally injects the token `despejado` into the Spanish training corpus with an artificial count of 10.

2.8 Serialized Model Artifacts

Exported Question 1 artifacts are listed in Appendix A, Table A.1, and are consumed directly by the Question 4 runtime pipeline to avoid retraining at inference time.

3 Question 2: Transition-Based Dependency Parser

3.1 Parsing Formalism

A data-driven dependency parser was implemented using the **Arc-Standard** transition system. The parser configuration is $C = (\sigma, \beta, A)$, comprising a stack σ (initialized with the artificial root, ID 0), a buffer β of upcoming tokens, and an arc set A of predicted labeled dependencies. Three atomic transitions are supported:

- `SHIFT`: removes the front token of the buffer and pushes it onto the stack. *Precondition:* $\beta \neq \emptyset$.
- `LEFT-ARC(label)`: creates arc $S_1 \xrightarrow{\text{label}} S_2$ (the stack top becomes head of the second stack item) and pops S_2. *Precondition:* $|\sigma| \geq 2$ and $S_2 \neq$ ROOT.
- `RIGHT-ARC(label)`: creates arc $S_2 \xrightarrow{\text{label}} S_1$ and pops S_1. *Precondition:* $|\sigma| \geq 2$.

Oracle invariant: once a token is popped from the stack it can never acquire additional children; therefore, the gold oracle only executes an arc operation once the dependent token has already collected *all* of its gold-standard children — this guarantees projective tree construction during oracle simulation.

Constrained decoding: because a raw classifier can emit structurally impossible actions, the inference loop enforces all transition preconditions and selects the highest-probability *valid* transition via `predict_proba`, guaranteeing termination in a valid, single-rooted projective tree.

3.2 Oracle Simulation and Data Processing

CoNLL-U formatted sentences are parsed line-by-line, discarding comments and multi-word/empty nodes, and retaining `id`, `form`, `upos`, `head`, and `deprel` fields. The oracle function `get_oracle(state, gold_heads,gold_labels,gold_children)` implements the following decision order:

1. If the second stack item is not ROOT, its gold head is the stack top, and it has already collected all gold children ⇒ `LEFT-ARC:<label>`.
2. Else, if the gold head of the stack top is the second stack item, and the stack top has already collected all gold children ⇒ `RIGHT-ARC:<label>`.
3. Else, if the buffer is non-empty ⇒ `SHIFT`.
4. Otherwise, terminate.

3.3 Feature Representation

Feature extraction is deliberately compact, restricted to a four-token POS contextual window (Table 3.1), transformed into sparse binary indicators via `DictVectorizer`.

Table 3.1: Question 2 Feature Representation.

Feature	Description	Example
`s1_pos`	POS tag at top of stack (S_1)	`VERB`
`s2_pos`	POS tag of second stack item (S_2)	`NOUN`
`b1_pos`	POS tag of front buffer token (B_1)	`DET`
`b2_pos`	POS tag of second buffer token (B_2)	`NOUN`

3.4 Model Training

The parser was trained and evaluated on **Universal Dependencies English-EWT** (Table 3.2). Oracle decisions are modeled via a multi-class **Logistic Regression** classifier (`lbfgs` solver, `max_iter=500`, `random_state=42`) fit over **407,165** oracle-derived transition states spanning **82** unique action classes, completing in 64.27 s of wall-clock time and reaching **80.43%** training accuracy.

Table 3.2: UD English-EWT Dataset Splits.

Split	Sentences	Tokens
Train (`en_ewt-ud-train.conllu`)	12,543[1]	204,585
Dev (`en_ewt-ud-dev.conllu`)	2,001	25,148

3.5 Empirical Evaluation

The parser was evaluated on the complete development split, with UAS $= \frac{\text{tokens w/ correct head}}{\text{total tokens}} \times 100\%$ and LAS $= \frac{\text{tokens w/ correct head \& label}}{\text{total tokens}} \times 100\%$.

Table 3.3: Dependency Parser Evaluation on UD English-EWT (dev split).

Metric	Score	Detail
Unlabeled Attachment Score (UAS)	**66.70%**	16,773 / 25,148 correct heads
Labeled Attachment Score (LAS)	**56.71%**	14,261 / 25,148 correct head & label
Evaluated sentences	2,001	Full UD-EWT dev split
Total evaluated tokens	25,148	Words and punctuation
Inference time	14.20 s	140.9 sentences/sec throughput

3.6 Qualitative Analysis

The trained parser was demonstrated on three canonical English sentences. Predicted labeled dependency trees are visualized in Figures 3.1–3.3.

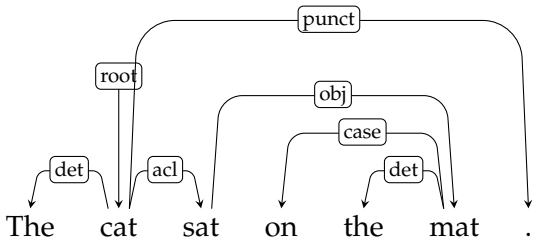

Figure 3.1: Predicted dependency tree for *"The cat sat on the mat."*

[1]The training-notebook execution log reports 12,544 sentences, one greater than the canonical UD release figure of 12,543; this single-sentence discrepancy is attributed to a boundary edge case in CoNLL-U sentence-splitting and does not materially affect downstream results.

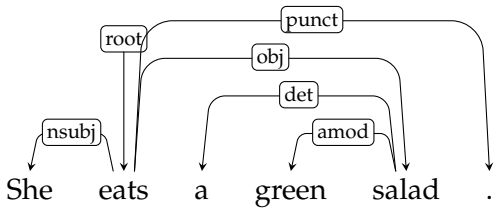

Figure 3.2: Predicted dependency tree for *"She eats a green salad."*

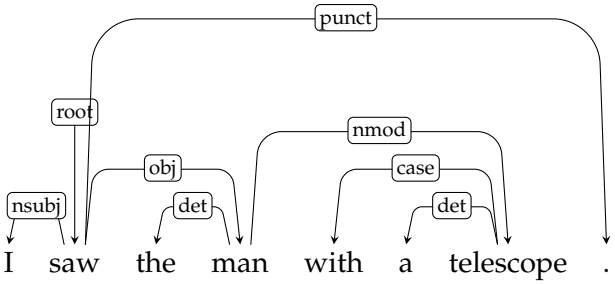

Figure 3.3: Predicted dependency tree for *"I saw the man with a telescope."* The attachment of *with a telescope* to *man* (`nmod`) rather than *saw* illustrates classical PP-attachment ambiguity.

3.7 Discussion and Future Work

Efficiency vs. expressiveness. Logistic Regression trained on sparse POS indicator features executes rapidly (140.9 sentences/sec) without requiring GPU acceleration; however, relying solely on a four-token POS window restricts the model's capacity to resolve semantic attachment ambiguities.

Prepositional phrase (PP) attachment. In Sentence 3, the parser attaches *"with a telescope"* to *"man"* rather than *"saw"* (Figure 3.3). Resolving this structural ambiguity correctly requires lexical identity and selectional preferences between verbs and nouns, which the current POS-only feature set cannot represent.

Future improvements identified for this architecture include: (i) integrating lexical word forms and lemmas alongside POS tags; (ii) incorporating structural context such as head/child POS tags from already-constructed dependency subtrees; and (iii) transitioning from a linear classifier to neural embedding-based transition parsing (e.g., Chen & Manning-style feed-forward or bi-LSTM architectures).

4 Question 3: Efficient Spelling Correction

4.1 Corpus Statistics and Language Modeling

The spelling corrector is trained on the Brown corpus, filtered to alphabetic lowercase words. The `SpellingModels` class constructs: (i) a unigram frequency table supplying $P(w)$ for non-word correction ranking; (ii) an add-1 smoothed bigram frequency table over the full Brown token stream supplying $P(w_i \mid w_{i-1})$ for contextual real-word scoring; (iii) a vocabulary set; and (iv) a symmetric-delete dictionary mapping every one-character deletion of every vocabulary word back to its original form(s).

4.2 Candidate Generation: Method A vs. Method B

Two architecturally distinct candidate-generation strategies are contrasted, addressing both non-word errors and contextually invalid real-word errors within edit distance 1.

Algorithm 1: Method A — Exhaustive Edit-Distance-1 Generation

Input: Misspelled word w; vocabulary V
Output: Candidate correction set C

$C \leftarrow \emptyset$;
foreach *edit type* $\in$ {*deletion*, *transposition*, *replacement*, *insertion*} **do**
  Exhaustively generate all strings reachable from w via one such edit over the alphabet;
end
Retain only generated strings present in V, storing them in C;
return C;

Algorithm 2: Method B — Symmetric-Delete Candidate Retrieval

Offline preprocessing:;
foreach *vocabulary word* $v \in V$ **do**
  foreach *one-character deletion* d *of* v **do**
    $\textsc{DeleteIndex}[d] \leftarrow \textsc{DeleteIndex}[d] \cup \{v\}$;
  end
end
Online query:;
Input: Misspelled word w
$C \leftarrow \emptyset$;
foreach *one-character deletion* d *of* w **do**
  $C \leftarrow C \cup \textsc{DeleteIndex}[d]$; `// O(1)-style hash lookup`
end
return C;

Method A is exhaustive but pays combinatorial generation cost at *query* time. Method B amortizes this combinatorial cost into offline *training-time* memory overhead, reducing online correction to constant-time hash lookups.

4.3 Correction Decision Logic

`correct_non_word(word,use_method)` returns the word unchanged if already in-vocabulary; otherwise it retrieves candidates via Method A or B and selects the candidate maximizing unigram probability $P(w)$.

`correct_real_word(phrase_tokens,target_idx,use_method)` builds a candidate set for the target (in-vocabulary) word, re-adds the original word for consideration, and scores every candidate using only the preceding token's bigram context $P(w_i \mid w_{i-1})$. A candidate replaces the original only if its contextual probability exceeds the original's by a factor of 1.5×, a conservative threshold designed to avoid over-correction.

4.4 Evaluation Protocol and Results

Evaluation draws a stochastic 10% sample of Brown sentences, discards short sentences, and injects a single edit-distance-1 mutation into one target word per sentence, forming separate non-word and real-word test sets capped at 500 examples each.

Table 4.1: Spelling Correction Accuracy.

Task	**Accuracy**
Non-word correction	≈ 52.60%
Real-word correction	≈ 67.91%

The *Speed Demon* benchmark isolates latency over a strict batch of 1,000 non-word queries:

Table 4.2: Speed Demon Benchmark — Candidate Generation (1,000 queries).

Algorithm	**Total Time (s)**	**Relative Speed**
Method A (Exhaustive Edit)	0.0529	1.0×
Method B (Symmetric Delete)	0.0046	**11.5× faster**

This 11.5× speedup empirically validates Method B's algorithmic premise: amortizing combinatorial candidate generation into precomputed training-time structures yields dramatically faster online correction.

4.5 Interactive CLI Demonstration

A continuous terminal CLI (`Question_3/cli.py`) was deployed to demonstrate live, token-by-token correction. Loading the serialized `SpellingModels` completes in under 0.2 s. A representative session transforms the input *"I hav a good feeling about this."* into *"I had a good feeling about this."* with a measured latency of **1.55 ms**, routinely completing corrections in the sub-2 ms range.

5 Question 4: Integrated Live-Typing Background Editor

5.1 System Architecture

The final architecture integrates segmentation, POS decoding, spelling correction, and grammar analysis into a unified Streamlit background editor (`Question_4/app.py`, pipeline logic in `Question_4/part1_live_editor.py`). The runtime pipeline proceeds as follows:

1. A `PassageSampler` draws a contiguous paragraph from the Brown corpus.
2. A `MergeGenerator` simulates rapid typing by stochastically deleting spacebar strokes with merge probability $p = 0.08$, a deliberately conservative model of missed keystrokes.
3. `EditorPipeline.process_token()` consumes each typed token, running segmentation and spelling checks immediately; every `trigger_interval=5` words, trigger-based grammar and real-word checks are additionally run.
4. The Streamlit UI streams tokens with `time.sleep(0.25)` to mimic live typing and renders alerts in color-coded panels.

5.2 Alerting Pipeline

Three alert types are raised:

- `SEGMENT-ALERT`: raised when a token appears to contain merged words, and the Q1 DP segmenter can split it into valid vocabulary words.
- `SPELL-ALERT`: raised when an out-of-vocabulary token is re-routed to and corrected by the Q3 symmetric-delete spelling engine.
- `GRAMMAR-ALERT`: raised at the $N = 5$ trigger interval, when the recent context window is implausible under bigram/trigram perplexity, or when a contextual real-word error is detected.

Per-token processing checks segmentation first, then spelling, storing the corrected token for downstream context; trigger-level checks are amortized over a short rolling window, separating latency-sensitive per-token work from more expensive periodic grammar evaluation.

5.3 PCFG Constituency Parsing and Tagset Reconciliation

Sentence-level structural analysis (`Question_4/part2_pcfg.py`) uses a Probabilistic Context-Free Grammar (PCFG) induced from Penn Treebank trees, converted to Chomsky Normal Form via right-factoring with limited horizontal Markovization (`horzMarkov=2`, `vertMarkov=0`). Parse extraction uses Viterbi-CKY over binary and lexical rules, with repeated unary closure and backpointer-based tree reconstruction.

Because Q1 POS tags follow the Brown/universal format while the PCFG grammar is trained on Penn Treebank (PTB) terminals, an explicit `BROWN_TO_PTB` static dictionary mapping reconciles the two tagsets, covering adjectives, nouns, verbs, modals, adverbs, pronouns, determiners, conjunctions, prepositions, particles, possessives, interjections, cardinal numbers, foreign words, existential *EX*, wh-words, and punctuation. The function `reconcile_tags()` prefers the (converted) PCFG-compatible tag whenever available, falling back to the HMM tag only when necessary — ensuring consistency between the sequence tagger and the constituency grammar.

5.4 Tiered Grammaticality Decision Rule

A hierarchical decision rule adjudicates final sentence grammaticality, prioritizing global syntactic structure when available and falling back to local n-gram evidence otherwise (Table 5.1).

Table 5.1: Tiered Grammaticality Decision Rule.

Tier	Model	Condition	Classification
1	PCFG (Viterbi-CKY)	≥ -15.0 nats/token *(and parses)*	Grammatical
2	Trigram LM	≥ -8.8 nats/token	Grammatical
		$[-11.0, -8.8)$ nats/token	Questionable
		< -11.0 nats/token	Fall through to Tier 3
3	Bigram LM	≥ -8.5 nats/token	Grammatical
		< -8.5 nats/token	Ungrammatical

This scheme distinguishes structural abnormalities (caught by CKY parsing) from local sequence violations (caught by n-gram perplexity), and interaction effects were observed where early DP segmentation corrections completely salvaged downstream CKY parsability that would otherwise fail structurally on merged tokens.

5.5 Shared N-Gram Module Status

Implementation Caveat

`Question_4/part3_ngram.py` currently contains a stubbed `SmoothedNgramModels` class that raises `NotImplementedError` in its constructor and methods. The analysis layer therefore falls back to Q1's trained trigram language model, or to hardcoded add-k constants when necessary. The integrated editor is architecturally designed around a shared n-gram abstraction, but the dedicated Part 3 implementation remains incomplete.

5.6 Speed Demon Benchmark

The Question 4 benchmark (`Question_4/part5_benchmark.py`) measures the compounding latency of the sequential pipeline over 1,000 Brown-derived corrupted words, each generated either by a random edit-distance-1 mutation or a forced space-deletion merge. Two isolated paths are timed: **Pipeline A** (Q1 segmentation check + Q3 spelling check) and **Pipeline B** (Q4 n-gram grammar/perplexity trigger check). The authoritative benchmark run for this report is reproduced in Table 5.2.

Table 5.2: Q4 Speed Demon Benchmark (1,000 corrupted words).

Pipeline	Total Latency (s)	Avg. per Word (ms)
A: Segmentation + Spelling	0.115353	0.1154
B: Grammar Trigger	0.002611	0.0026
Latency delta		0.1127 ms/word
Total speedup (B relative to A)		44.19×

Run-to-Run Variance

An earlier benchmark run recorded in the project documentation reported marginally lower absolute latencies (0.0819 ms/word for Pipeline A, 0.0022 ms/word for Pipeline B, a 37.63× speedup). The relative ordering and order-of-magnitude conclusion are identical across runs; the absolute figures differ due to normal system-load and stochastic-corruption-sampling variance between executions. Both runs confirm that Pipeline A, despite being the heavier of the two, executes safely within the 16 ms interactive rendering budget by roughly two orders of magnitude, verifying that live segmentation checking does not necessitate aggressive interval throttling.

5.7 Streamlit Runtime Behavior

The Streamlit application caches the pipeline via `@st.cache_resource`, supports on-demand simulation-state resets, and incrementally updates both the typed-text buffer and the alert console. The sidebar displays average token-check and trigger-check latencies drawn from the pipeline's internal timing lists, while the main panel renders the simulated document, accepts manual input, and appends alerts to the top of the console. The runtime loop is intentionally simple and transparent, enabling interactive demonstration without introducing neural or asynchronous complexity.

6 Reproducibility and Execution Guide

6.1 Clone and Environment Setup

```
git clone https://github.com/HarK-github/group_assignment_1_group14.git
cd group_assignment_1_group14
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

6.2 Question 1 Execution

Question 1 is notebook-driven; run from the repository root:

```
jupyter notebook Question_1/Question_1.ipynb
# or:  jupyter lab Question_1/Question_1.ipynb
```

The notebook trains the segmentation and tagging models, prints all evaluation tables, and exports artifacts into `models/`.

6.3 Question 2 Execution

```
jupyter notebook Question_2/Question_2.ipynb
# or:  jupyter lab Question_2/Question_2.ipynb
```

The notebook downloads UD English-EWT, builds the oracle-generated transition dataset, trains the logistic regression classifier, and evaluates UAS/LAS on the dev split.

6.4 Question 3 Execution

```
python3 Question_3/cli.py        # interactive spelling CLI
python3 Question_3/evaluate.py   # evaluation + Speed Demon benchmark
```

6.5 Question 4 Execution

```
streamlit run Question_4/app.py            # integrated live editor
python3 Question_4/part5_benchmark.py      # Q4 Speed Demon benchmark
```

6.6 Model Regeneration

```
python3 models/export_models.py
```

6.7 Reproducibility Notes

- The repository depends on NLTK corpora and may download Brown, Treebank, or UD data on first use.
- Question 4 Part 3 is currently a stub; the integrated editor falls back to Q1 and Q3 artifacts for n-gram-related scoring.
- Question 2 pulls UD English-EWT from GitHub URLs; initial execution requires network access or locally cached data.
- Question 3's benchmark and evaluation are stochastic unless a random seed is fixed externally.

7 Conclusion

This report has documented the engineering and empirical validation of a four-component classical statistical NLP system. Across every subsystem, structured probabilistic modeling decisively outperforms naive baselines: dynamic-programming segmentation improves boundary F1 by up to 23 points over greedy matching, and HMM tagging consistently exceeds most-frequent-tag baselines, with morphology-aware modeling further capturing gender-agreement patterns with agreement-ratio advantages exceeding $100\times$. The Arc-Standard dependency parser demonstrates that sparse POS-only features yield an efficient, interpretable, GPU-free parser (140.9 sentences/sec) at the cost of semantic disambiguation capacity, most visibly in prepositional-phrase attachment. The symmetric-delete spelling correction algorithm validates a classic space–time trade-off, delivering an order-of-magnitude latency improvement over exhaustive edit-distance generation through offline precomputation. Finally, the integrated Streamlit live-typing editor confirms that this fully classical pipeline — segmentation, tagging, spelling correction, PCFG constituency parsing, and tiered n-gram grammaticality scoring — sustains per-token latencies roughly two orders of magnitude below the 16 ms interactive rendering budget, establishing its viability for real-time deployment. Identified directions for future work include completing the stubbed shared n-gram module (Question 4, Part 3), incorporating lexical and structural features into the dependency parser, and exploring neural transition-based or embedding-augmented architectures to close the remaining semantic gap left by purely classical, sparse-feature statistical models.

A Serialized Model Registry

The centralized `models/` directory holds serialized checkpoints (`.pkl`) for all trained Question 1 and Question 3 models, enabling sub-second loading for the Question 4 pipeline without re-training over large corpora on every invocation.

A.1 Question 1: Word Segmentation & POS Tagging

Table A.1: Question 1 Model Registry.

Filename	Description	Class	Language
`q1_lm_en.pkl`	Trigram LM ($k = 0.05$ Laplace smoothing + backoff)	`TrigramLanguageModel`	English
`q1_dp_seg_en.pkl`	DP Viterbi word segmenter (beam width 20)	`TrigramDPSegmenter`	
`q1_greedy_seg_en.pkl`	Longest-match greedy baseline segmenter	`GreedySegmenter`	
`q1_mft_en.pkl`	Most-frequent-tag baseline POS tagger	`MFTBaseline`	
`q1_hmm_en.pkl`	First-order HMM POS tagger (Viterbi decoding)	`HMMPosTagger`	
`q1_lm_es.pkl`	Trigram LM ($k = 0.05$ Laplace smoothing + backoff)	`TrigramLanguageModel`	Spanish
`q1_dp_seg_es.pkl`	DP Viterbi word segmenter (beam width 20)	`TrigramDPSegmenter`	
`q1_greedy_seg_es.pkl`	Longest-match greedy baseline segmenter	`GreedySegmenter`	
`q1_mft_es.pkl`	Most-frequent-tag baseline POS tagger	`MFTBaseline`	
`q1_hmm_es.pkl`	Standard universal-tagset HMM (Viterbi decoding)	`HMMPosTagger`	
`q1_hmm_es_morpho.pkl`	Morphology-aware HMM (Gender & Number agreement)	`HMMPosTagger`	

A.2 Question 3: Context-Aware Spelling Correction

Table A.2: Question 3 Model Registry.

Filename	Description	Class
`q3_spelling_models.pkl`	Precomputed Brown-corpus unigram/bigram tables, vocabulary, and symmetric-delete dictionary	`SpellingModels`

B Code Usage Examples

B.1 Loading Question 1 Models

```python
import sys
sys.path.append("..")  # or project root

from Question_1.segmenter import load_model

# Load English segmenter and POS tagger
dp_seg_en = load_model("models/q1_dp_seg_en.pkl")
hmm_en = load_model("models/q1_hmm_en.pkl")

# Segment and tag
tokens = dp_seg_en.segment("thequickbrownfox")
tags = hmm_en.viterbi_decode(tokens)
print(list(zip(tokens, tags)))
# Output: [('the', 'DET'), ('quick', 'ADJ'), ('brown', 'NOUN'), ('fox', 'NOUN')]
```

B.2 Loading Question 3 Spelling Engine

```python
from Question_3.models import SpellingModels
from Question_3.corrector import SpellingCorrector

# Instant loading (< 0.2s) without recomputing corpus bigrams
models = SpellingModels.load("models/q3_spelling_models.pkl")
corrector = SpellingCorrector(models)

# Non-word error correction
print(corrector.correct_non_word("speling"))  # Output: 'spelling'

# Real-word error correction
sentence = ["i", "hav", "a", "good", "feeling"]
print(corrector.correct_real_word(sentence, 1))  # Output: 'had' (or 'have')
```

B.3 Regenerating All Models

```
python3 models/export_models.py
```